Quantitative Trading System Architect Prompt

Reverse-Engineering Ross Cameron's Discretionary Momentum Methodology
into a Deterministic, Production-Ready Trading System

Revised Edition — with quantitative thresholds, MVP prioritization, backtest realism, and acceptance criteria

Context & Instructions

The agent must act as a **quantitative trading system architect**, not as a coder. Phase 1 produces a rigorous specification that converts Ross Cameron's discretionary approach into objective, testable rules **before any implementation begins**.

Give the entire prompt below to the agent. Do not omit sections.

1. Objective

You are an experienced quantitative trader, software architect, and Python engineer. Your task is **NOT** to build the software yet.

Reverse-engineer Ross Cameron's discretionary momentum trading methodology into a fully specified, deterministic trading system that can later be implemented in Python. The final product is assumed to be a professional trading platform for US equities connected to Interactive Brokers (IBKR).

Output for this phase: a complete Product Requirements Document (PRD), software architecture, implementation roadmap, and technical specifications.

2. Core Principle

Ross Cameron teaches discretionary momentum trading. Many concepts are subjective. Your first responsibility is to identify every subjective decision and convert it into explicit measurable rules.

Whenever a rule cannot be objectively defined:

- Explain why it cannot be objectively defined
- Propose multiple deterministic alternatives with concrete numeric parameters where possible
- Discuss the tradeoffs of each alternative
- Recommend one implementation and mark it clearly as an assumption

Do NOT leave discretionary logic unspecified. Every decision point must have a rule or an explicitly documented assumption.

3. Must-Define Quantitative Thresholds

Before writing any other specification, the agent **must** propose concrete default values (or ranges) for the following parameters. Mark each as High / Medium / Low confidence and cite the Ross source material (or note the absence of a source).

Parameter	Why it matters	Example defaults to propose
Minimum Gap %	Core filter for gappers	e.g. ≥ 4%, ≥ 8%, ≥ 10%
Relative Volume (RVOL)	Momentum / interest proxy	e.g. ≥ 2×, ≥ 5× average
Float	Low-float preference	e.g. ≤ 10M, ≤ 20M, ≤ 50M shares
Price range	Liquidity & PDT constraints	e.g. $1–$20, $2–$50
Average Daily Volume	Liquidity filter	e.g. ≥ 500k, ≥ 1M shares
Premarket volume	Early interest signal	Absolute and/or vs prior day
Max extension from VWAP / HOD	Avoid chasing	% or ATR multiples
Stop distance	Risk per trade	% of price, ATR, or candle low
Profit target / R-multiple	Exit logic	1R, 2R, trailing, or partials

Max risk per trade	Position sizing	e.g. 0.25%–1% of equity
Daily loss limit	Account protection	e.g. 2%–3% of equity
Max open positions	Concentration risk	e.g. 1–4 concurrent

For each threshold the agent must also state: (a) the source in Ross's material if any, (b) sensitivity — how much performance may change if the value is altered, and (c) whether it should be a user-configurable parameter in the final system.

4. Trading Style — Required Setups

The system must support all major components of Ross Cameron's strategy listed below. If additional recurring setups appear in the source material and are suitable for deterministic implementation, include them and mark confidence.

- Gap scanner
- Low float momentum stocks
- Relative volume
- News catalysts
- Premarket gappers
- Opening Range Breakout
- First 1-min candle new high
- Five-minute breakout
- ABCD setups
- Bull flags
- Flat-top breakouts
- Micro pullbacks
- VWAP reclaim
- Halt breakouts
- High-of-day breakouts
- Whole-dollar breakouts
- Momentum continuation
- Pullback entries
- Scaling into winners
- Scaling out
- Risk management
- Daily loss limits
- Maximum number of losses
- Position sizing
- Trade journaling
- Statistics

5. Market, Broker & Language

Market: US equities only.

Broker: Interactive Brokers (IBKR). All execution via the official IBKR API. Explicitly list required market-data subscriptions and note approximate monthly cost implications.

Language: Python. Modular, production-ready architecture.

6. Phase 1 Deliverables

Produce a complete PRD covering the areas below. Every section must be concrete enough that implementation can begin with minimal follow-up questions.

6.1 Functional Requirements

Document every feature, screen, workflow, and user interaction. Include primary and secondary user journeys (scanner → setup identification → entry → management → exit → journal).

6.2 Non-functional Requirements

Address: performance, reliability, latency (order-to-exchange and data-to-signal), security, logging, audit trail, configuration management, testing strategy, deployment, and scalability.

6.3 Trading Rules Specification

For every setup include all of the following:

- Entry criteria (boolean + numeric)
- Exit criteria
- Stop placement (exact rule)
- Profit targets / R-multiples
- Risk calculations & position sizing
- Filters (pre- and post-signal)
- Invalidation rules
- Required confirmations
- Optional confirmations
- Edge cases
- Worked examples (with numbers)
- Known false-signal patterns

6.4 Scanner Specification

Design a scanner for Ross-style momentum stocks. For each filter explain why it exists, the proposed default threshold (see Section 3), and whether it is hard or soft.

- Gap %
- Relative volume
- Float
- Market cap
- Price range
- Average daily volume
- Premarket volume
- News / catalyst
- Volatility
- Circuit breakers
- Recent halts
- Institutional ownership (if useful)
- Short interest (if useful)
- Liquidity / spread requirements

6.5 Market Data Requirements

Specify exact data needs and quality requirements:

- Real-time Level I
- Level II / depth
- Time & Sales
- Historical bars (1-min, 5-min, daily)
- Premarket & after-hours
- News feeds (sources & latency)
- Corporate actions & splits
- Halt / LULD status
- Trading calendar
- Required IBKR subscriptions & cost

Explicitly address data quality issues common to low-float names (missing ticks, inconsistent premarket volume, halt resumption gaps).

6.6 Trading / Execution Engine

Design the full order lifecycle. Cover:

- Market, Limit, Stop, Stop-limit
- Bracket / OCO orders
- Partial fills handling
- Slippage model & assumptions
- Order retries & backoff
- Rejected-order handling
- Connection-failure recovery
- Duplicate-order protection
- Pre-trade risk validation
- Post-fill reconciliation

6.7 Risk Management Engine (Hard Rules)

Risk rules must be enforceable at defined points in the order and position lifecycle. For every hard rule specify: (1) the exact condition, (2) the enforcement point (pre-order check, post-fill, end-of-day, continuous), and (3) the action on violation (reject order, flatten position, lock account, alert only).

- Max risk per trade (% equity)
- Dynamic position sizing formula
- Max daily loss (hard stop)
- Max drawdown (session / multi-day)
- Max open positions
- Max sector / correlated exposure
- Max buying power usage
- Emergency kill switch
- Loss-streak lockout
- PDT / pattern-day-trader checks
- Trading-hours lockout
- Hard rules that cannot be bypassed by the user

6.8 Backtesting Framework — Realism Requirements

Professional-grade backtesting is required. In addition to standard metrics, the design **must** address the following realism problems that are acute for low-float momentum strategies:

- Partial fills and liquidity constraints on thin names
- Realistic slippage model (spread + impact, especially on halt resumptions)
- Halt / LULD simulation and resumption gap handling
- Look-ahead bias controls (especially relative volume and news timestamps)
- Premarket and opening-auction modeling
- Corporate-action and split adjustment correctness
- Walk-forward and out-of-sample protocol
- Monte Carlo / bootstrap of trade sequence for drawdown confidence

Required metrics: expectancy, Sharpe (or similar), profit factor, max drawdown, average winner / loser, holding time distribution, win rate, risk-adjusted returns, and trade-count sufficiency checks.

6.9 Architecture

Recommend a modular architecture. Each component must have clearly defined responsibilities, interfaces, and data contracts:

- Scanner
- Data ingestion
- Indicators / feature store
- Strategy engine
- Risk engine
- Execution engine
- Portfolio manager
- Trade journal
- Analytics
- Backtester
- Configuration
- Database
- Logging & audit
- Monitoring
- Notification system

6.10 Database Design

Recommend schema for:

- Market data (bars, ticks if needed)
- Orders & executions
- Positions
- Closed trades
- Performance snapshots
- Configurations & strategy parameters
- Logs / audit trail
- Watchlists
- Journal entries

6.11 User Interface

Desktop application. Recommend framework and layout. Cover at minimum:

- Trading dashboard
- Scanner view
- Charts with overlays
- Orders & positions
- Risk panel (live limits)
- Statistics / equity curve
- Backtesting interface
- Settings / parameter editor
- Trade journal
- Notifications / alerts

6.12 Development Roadmap & MVP Definition

Split work into milestones. **Explicitly define an MVP** that enables paper or live trading of the highest-confidence setups with full risk controls, even if the GUI and advanced analytics are incomplete.

Phase 0	Research & source-material extraction + threshold proposals (Section 3)
Phase 1	Core architecture, data contracts, configuration
Phase 2	Market-data ingestion (incl. premarket) + basic quality checks
Phase 3	Scanner with all hard filters
Phase 4	Strategy engine — highest-confidence setups only
Phase 5	Execution engine + pre-trade risk checks
Phase 6	Full risk engine (hard rules, kill switch, lockouts)
MVP Gate	Paper-trade ready: scanner + 2–3 setups + risk + journal (no fancy GUI required)
Phase 7	Backtesting with realism requirements (Section 6.8)
Phase 8	Desktop GUI
Phase 9	Parameter optimization & walk-forward
Phase 10	Production hardening, monitoring, deployment

Estimate complexity, dependencies, and risk for each phase. Call out any phase that depends on unresolved discretionary concepts.

6.13 Assumptions

Whenever information is missing: do not guess silently. Explicitly state the assumption, recommend alternatives, and explain consequences for system behavior and performance.

6.14 Future Phases (Do Not Design Yet)

Reserve extension points only. Do not design AI features in this phase. Possible later items:

- AI-assisted trade review
- Natural-language trade explanations
- Strategy optimization assistance
- News summarization
- Pattern ranking
- Journal insights

7. Strategy Validation and Discretion Analysis

Before finalizing any specification, perform a formal analysis of Ross Cameron's methodology. The goal is transparency about where the automated system diverges from discretionary trading.

7.1 Source Review

Review publicly available educational material (books, videos, webinars, blog posts, documented examples). Distinguish consistently taught principles from isolated examples. Do not treat every example as a strict rule.

7.2 Rule Extraction

For every trading concept:

- Identify the original teaching and cite the source where possible
- Translate into one or more deterministic rules with numeric parameters
- State confidence level (High / Medium / Low) that the rule reflects the original methodology
- Explain assumptions and known approximations

7.3 Discretion Analysis

List every place Ross relies on trader judgment. For each discretionary element:

- Explain why it is subjective
- Describe how experienced traders typically interpret it
- Propose multiple objective implementations (with numbers where possible)
- Discuss advantages / disadvantages of each
- Recommend the implementation most suitable for systematic trading

Examples of discretionary elements that must be addressed:

- Reading momentum / tape
- Assessing candle quality
- Evaluating volume strength
- Recognizing "clean" charts
- Judging whether a move is extended
- Deciding if a pullback is healthy
- Evaluating market sentiment
- Deciding if a stock is "too obvious"
- Conviction / setup quality scoring
- When to skip an otherwise valid setup

7.4 Validation Matrix

Produce a table with the columns below. Every major component must appear. Example row is provided to set the expected level of concreteness:

Strategy Concept	Ross Teaching	Deterministic Rule	Conf.	Assumptions	Alternatives
Relative Volume filter	Prefers stocks trading 'much higher than average' volume	RVOL ≥ 5× 50-day ADV at signal time; computed on 1-min bars	Medium	5× is a common community proxy; Ross rarely states an exact multiple	3× or 10×; volume z-score; premarket-only RVOL
…	…	…	…	…	…

7.5 Known Limitations

Explicitly identify parts of the methodology that cannot be faithfully automated without human judgment. For each limitation: explain why it exists, likely performance impact, mitigation via deterministic rules, and whether it is a candidate for future AI assistance (Phase 2) — without designing the AI features.

7.6 Confidence Report

Categorize every component:

High Confidence — Can be implemented objectively with minimal ambiguity.

Medium Confidence — Requires reasonable assumptions but is still suitable for systematic use.

Low Confidence — Relies heavily on discretion; may not closely replicate the original methodology without further research or future AI assistance.

8. Phase 1 Acceptance Criteria

The PRD is considered complete only when all of the following are true:

- Every setup in Section 4 has fully specified entry, exit, stop, target, and invalidation rules with numeric parameters where applicable.
- Section 3 thresholds are populated with proposed defaults, confidence ratings, and source notes.
- Every identified discretionary element has at least one recommended deterministic implementation and a documented alternative.
- The Validation Matrix covers all major components and contains no empty 'Deterministic Rule' cells.
- Risk rules specify enforcement point and violation action; at least the daily loss limit and max risk-per-trade are hard (non-bypassable).
- Backtest design explicitly addresses the realism items in Section 6.8.
- An MVP scope is clearly defined and mapped to the roadmap (scanner + highest-confidence setups + risk + basic journal).
- All assumptions are listed in one place with consequences.
- A software engineer unfamiliar with Ross Cameron could begin implementation of the MVP without needing to ask clarifying questions about trading logic.

9. Expected Output Characteristics

Produce a professional engineering design document suitable for a software team. Be exhaustive. Challenge assumptions. Identify hidden complexities and risks. Recommend best practices from professional algorithmic trading systems.

The document must be detailed enough that implementation of the MVP can begin with minimal ambiguity. Explicitly compare extracted rules against Ross Cameron's publicly available material and note where the system is an approximation of discretionary trading.

The objective is not merely to replicate Ross Cameron's strategy, but to transform it into a transparent, auditable, testable, and production-ready quantitative trading system while clearly documenting every point of divergence from discretionary judgment.